

19. sim 用車体状態 publisher

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	4
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	6
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	6
6.2 名前、代入、型、参照	6
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	7
6.5 例外、try/finally、with、資源解放	7
6.6 class、独自クラス、インスタンス、self、継承	7
6.7 先頭アンダースコア、__init__、名前付けの作法	8
6.8 list、dict、deque、NumPy 配列、pandas DataFrame	8
6.9 rclpy、rcl、rmw、DDS/RTPS の層	8
6.10 ROS graph、Node、topic、service、action、parameter	9
6.11 callback、timer、Executor、spin、Callback Group	9
6.12 rqt_graph、ros2 CLI、rosviz をいつ使うか	9
6.13 車両力学、電力収支、効率 map	10
6.14 SoC、内部抵抗、端子電圧、温度、MHE	10
6.15 天候、route、補間、時刻、単位	11

6.16 制御、状態、入力、モデル予測制御 MPC	11
7 関数・クラスを上から順に解説	11
7.1 L22 クラス SolarStateNode	11
7.2 L23 関数 SolarStateNode.__init__	12
7.3 L180 関数 SolarStateNode._on_speed_cmd	16
7.4 L183 関数 SolarStateNode._forecast_row	17
7.5 L191 関数 SolarStateNode._route_value	18
7.6 L199 関数 SolarStateNode._step	19
7.7 L265 関数 main	21
7.8 設定値と入力パラメータ	22
7.9 ROS topic I/O	23
8 処理の流れ	23
9 このファイルの位置づけ	23

1 対象

項目	内容
ファイル	mpc_solarcar/solar_state_node.py
ソース SHA-256	d9c72bcecf7b5d6f0594d4c9029ba3837a9f3b0222e47e693729d581d99e4024
種別	Python
区分	runtime node
役割	sim モードで planner 指令速度から速度、距離、電池、PV、altitude を模擬 publish する。
起動文脈	simulation launch における擬似車両。

2 このファイルを読む理由

sim モードで planner 指令速度から速度、距離、電池、PV、altitude を模擬 publish する。

- SolarCarModel を直接生成する。
- /planner/speed_cmd を受けて /vehicle/* を出す。

3 呼び出し関係

3.1 どこから呼ばれるか

- launch/solarcar_sim.launch.py

3.2 次に読むと理解が進むファイル

- mpc_solarcar/model.py
- mpc_solarcar/mpc_node.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/model.py
- mpc_solarcar/path_utils.py
- mpc_solarcar/route_utils.py
- mpc_solarcar/schedule_utils.py
- mpc_solarcar/signal_utils.py
- mpc_solarcar/solar_profile.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
from .solar_profile import get_path, get_section, load_profile, require_csv_data_rows

class SolarStateNode(Node):
    def __init__(self) -> None:
        super().__init__("solar_state_node")
        self.declare_parameter("profile_yaml", "")
        self.declare_parameter("forecast_csv", "inputs/forecast_10min.csv")
        self.declare_parameter("route_profile_csv", "")
        self.declare_parameter("drive_schedule_yaml", "")
        self.declare_parameter("drive_eff_map", "")
        self.declare_parameter("regen_eff_map", "")
        self.declare_parameter("rint_map", "")
        self.declare_parameter("panel_eff_map", "")
        self.declare_parameter("mppt_eff_map", "")
        self.declare_parameter("drive_map_eco", "")
        self.declare_parameter("drive_map_power", "")
        self.declare_parameter("regen_map_eco", "")
        self.declare_parameter("regen_map_power", "")
        self.declare_parameter("ocv_soc_map", "")
        self.declare_parameter("params_yaml", "")
        self.declare_parameter("forecast_time_mode", "auto")
```

5 主要構造の読み取り

主要クラスは SolarStateNode。主要関数は main。ROS パラメータ宣言は 26 件。ROS I/O は publisher 8 件、subscription 1 件。

5.1 主要クラス・関数

行	種別	名前	補足
22	class	SolarStateNode	bases: Node
23	function	__init__	args: self
180	function	_on_speed_cmd	args: self, msg
183	function	_forecast_row	args: self, now_utc
191	function	_route_value	args: self, field, default
199	function	_step	args: self
265	function	main	

5.2 ファイルを上から読んだときの定義順

行	内容
22	クラス SolarStateNode を定義する。
265	関数 main を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from__future_ _importannotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>importmath</code>	Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での使用位置は少ないか、間接利用である。
4	<code>importtime</code>	wall/monotonic time に基づく周期制御や freshness 判定を行うため。このファイル内での使用位置は少ないか、間接利用である。
5	<code>fromdatetimeimportdatetime, timedelta,timezone</code>	UTC 時刻や相対時間を扱うため。このファイル内での主な使用位置は L118, L122, L183, L209。
7	<code>importnumpyasnp</code>	数値配列、clip、補間、統計計算を行うため。このファイル内での主な使用位置は L185, L186, L188, L240。
8	<code>importpandasaspd</code>	CSV や時刻列の読み込み・整形・再サンプリングに使うため。このファイル内での主な使用位置は L99, L105, L107。
9	<code>importrcclpy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L59, L60, L61, L62, L63, L64, L65, L66, ...。
10	<code>fromrcclpy.nodeimportNode</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L22。
12	<code>fromstd_msgs. msgimportFloat32</code>	Float32、Bool、String などの標準 ROS message で値を publish または subscribe するため。このファイル内での主な使用位置は L168, L169, L170, L171, L172, L173, L174, L175, ...。
14	<code>from.modelimportParams, SolarCarModel</code>	車体物理・電気モデル本体から Params, SolarCarModel を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/model.py。このファイル内での主な使用位置は L128, L132。
15	<code>from.path_ utilsimportresolve_path</code>	ROS share / 相対パス解決から resolve_path を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/path_utils.py。このファイル内での主な使用位置は L87, L95, L103, L129, L130, L131, L133, L134, ...。
16	<code>from.route_ utilsimportinterpolate_ profile</code>	route_utils.py から interpolate_profile を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/route_utils.py。このファイル内での主な使用位置は L195。
17	<code>from.schedule_ utilsimportDriveSchedule</code>	schedule_utils.py から DriveSchedule を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/schedule_utils.py。このファイル内での主な使用位置は L103。
18	<code>from.signal_ utilsimportSmoothRateLimiter</code>	signal_utils.py から SmoothRateLimiter を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/signal_utils.py。このファイル内での主な使用位置は L158。

19	from.solar_ profileimportget_path, get_section,load_profile, require_csv_data_rows	profile YAML 読込と検証から get_path, get_section, load_profile, require_csv_data_rows を読み込み、この ファイルの内部処理を分担させるため。実体ファイ ルは mpc_solarcar/solar_profile.py。このファイル内 での主な使用位置は L54, L55, L56, L59, L60, L61, L62, L63, ...。
----	---	---

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、`*args`、`**kwargs`は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが`self.z`を更新すればオブジェクトの状態が残るが、単に`z`を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。`from .model import SolarCarModel`の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。`def`や`class`は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の`console_scripts`は、端末で使う実行可能名と`package.module:function`を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。`try/except`は想定した異常を処理し、`finally`は成功・失敗にかかわらず後始末を行う。

`with open(...) as f:`は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

`except Exception: pass`はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

`class MPCNode(Node)`は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 `self` として ‘`__init__`’ へ渡す。‘`self`’ は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘`super().__init__(...)`’ は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘`create_publisher`’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、`__init__`、名前付けの作法

‘`self._step_solar`’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘`__init__`’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘`_`’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘`_base_csv`’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 ‘(N)’、候補集団なら ‘(population, N)’となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

6.9 rclpy、rcl、rmw、DDS/RTSPS の層

rclpy は Python 利用者向け ROS 2 client library である。利用者の Node、publisher、subscriptionなどを Python API として提供し、その下で共通 C 層の rcl を利用する。

本プロジェクトのPythonコード -> rclpy -> rcl -> rmw -> DDS/RTSPS実装 -> ネットワークまたは同一PC内通信

rcl は言語に依存しない共通 ROS 機能を提供する C API、rmw は ROS 2 と具体的 middleware 実装の境界である。DDS/RTSPS 側が探索、serialize、publish/subscribe、request/replyなどを担う。executorの実行モデルは rcl だけで完結せず client library 側にも実装される。

根拠資料

- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.10 ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、action などの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [ROS 2 Humble 公式: Topics, Services, Actions](#)
- [ROS 2 Humble 公式: Parameters](#)

6.11 callback、timer、Executor、spin、Callback Group

callback は、メッセージ受信、timer 満了、service request などのイベントが成立した後で Executor から呼ばれる関数である。登録時に 'self._on_speed' と括弧なしで渡すのは、今実行せず後で呼ぶ関数オブジェクトを渡すためである。

Executor は実行可能になった callback を見つけ、Callback Group の条件を確認して実行する。'spin()' は終了要求まで待機・dispatch を続けるため、通常運転中は main の次の行へ戻らない。

MutuallyExclusiveCallbackGroup は同じ group 内の callback を同時実行させない。別 group 間は Multi-ThreadedExecutor で並行実行し得る。group を分けただけでは共有属性への完全な排他にはならないため、複数 group が同じ状態を読む・書く場合は設計確認が必要である。

```
DDS受信またはtimer満了 -> wait setでready -> Executorが選択 -> Callback Groupが許可 -> worker threadがcallback実行 -> 終了後Executorへ戻る
```

根拠資料

- [ROS 2 公式: Executors](#)
- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.12 rqt_graph、ros2 CLI、rosviz をいつ使うか

rqt_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。

```
ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```

rosvbag2 は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。

```
ros2 bag record -o outputs/bags/preflight \
  /vehicle/s_km /vehicle/speed_kmh /vehicle/batt_soc \
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status

ros2 bag info outputs/bags/preflight
ros2 bag play outputs/bags/preflight --clock
```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

根拠資料

- ROS 2 Humble 公式: [rqt_graph](#)
- ROS 2 Humble 公式: [Recording and playing back data](#)
- ROS 2 Humble 公式: [Understanding nodes](#)

6.13 車両力学、電力収支、効率 map

$$F_{\text{aero}} = \frac{1}{2} \rho C_d A (v + v_{\text{wind}})^2, \quad F_{\text{roll}} \approx mg C_{rr} \cos \theta, \quad F_{\text{grade}} = mg \sin \theta$$

車輪機械 power は概ね ‘ $P_{\text{mech}} = F_{\text{total}} * v + P_{\text{inertia}}$ ’ で、駆動時と回生時で異なる効率 map を通して DC 側 power へ変換する。map は速度・torque などを軸にした実測または同定 table であり、範囲外の補間・clip 規則もモデルの一部である。

$$P_{\text{pack}} = P_{\text{drive,dc}} - P_{\text{regen,dc}} + P_{\text{aux}} - P_{\text{pv}}$$

符号規約は必ずコードで確認する。本プロジェクトでは正の pack power を放電側として SoC を減らす処理が中心であり、発電と回生は pack 負荷を下げる方向に働く。

6.14 SoC、内部抵抗、端子電圧、温度、MHE

$$V = V_{\text{oc}}(z, T) - IR_{\text{total}}(z, T, I), \quad z_{k+1} = z_k - \frac{\eta(I, T) I \Delta t}{Q_{\text{eff}}}$$

SoC は直接完全には観測できないため、電流積算、OCV、端子電圧、温度、容量、効率を組み合わせる。内部抵抗は SoC、温度、電流方向で変わり、発熱と電圧制約の両方へ影響する。

MHE は有限時間窓の状態と観測誤差をまとめて最適化し、SoC や温度を推定する。古い測定や欠損値を同じ重みで使うと推定が壊れるため、timestamp freshness と観測可用性を入口で確認する。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

6.15 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻 t と距離 s が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 `timezone`、`naive datetime` を混ぜると数時間のずれが発生する。入力で `timezone` を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

6.16 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが `receding horizon` である。

予測モデル、目的関数、制約、ホライズン、`solver`、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

7 関数・クラスを上から順に解説

7.1 L22 クラス `SolarStateNode`

- 行範囲: L22–L262
- 所属: `module` 直下
- 定義: `SolarStateNode(bases=Node)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- class 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 関数 `__init__` を定義する。
2. 関数 `_on_speed_cmd` を定義する。
3. 関数 `_forecast_row` を定義する。
4. 関数 `_route_value` を定義する。
5. 関数 `_step` を定義する。

代表コード断片

```
class SolarStateNode(Node):
    def __init__(self) -> None:
        super().__init__("solar_state_node")
        self.declare_parameter("profile_yaml", "")
        self.declare_parameter("forecast_csv", "inputs/forecast_10min.csv")
        self.declare_parameter("route_profile_csv", "")
        self.declare_parameter("drive_schedule_yaml", "")
        self.declare_parameter("drive_eff_map", "")
        self.declare_parameter("regen_eff_map", "")
        self.declare_parameter("rint_map", "")
        self.declare_parameter("panel_eff_map", "")
        self.declare_parameter("mppt_eff_map", "")
        self.declare_parameter("drive_map_eco", "")
        self.declare_parameter("drive_map_power", "")
        self.declare_parameter("regen_map_eco", "")
        self.declare_parameter("regen_map_power", "")
        self.declare_parameter("ocv_soc_map", "")
        self.declare_parameter("params_yaml", "")
        self.declare_parameter("forecast_time_mode", "auto")
        self.declare_parameter("forecast_time_tz", "UTC")
        self.declare_parameter("forecast_start_time_utc", "")
        self.declare_parameter("publish_rate_hz", 2.0)
        self.declare_parameter("init_speed_kmh", 45.0)
        self.declare_parameter("soc0", 0.95)
        self.declare_parameter("Tb0", 30.0)
        self.declare_parameter("s0_kmh", 0.0)
        self.declare_parameter("filter_tau_sec", 1.0)
        self.declare_parameter("accel_limit_kmhps", 1.2)
        self.declare_parameter("decel_limit_kmhps", 3.5)

        profile_yaml = str(self.get_parameter("profile_yaml").value or "").strip()
        if profile_yaml:
            profile_path, cfg = load_profile(profile_yaml)
            sim_cfg = get_section(cfg, "simulation")
            runtime_cfg = get_section(cfg, "runtime")
        ...
```

7.2 L23 関数 SolarStateNode.__init__

- 行範囲: L23–L178

- 所属: SolarStateNode
- 定義: `__init__(self) -> None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Parameter`, `Params`, `SmoothRateLimiter`, `SolarCarModel`, `__init__`, `astimezone`, `create_publisher`, `create_subscription`, `create_timer`, `declare_parameter`, `float`, `from_yaml`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `_profile_path`, `cfg`, `drive_schedule_yaml`, `key`, `model_cfg`, `params_yaml`, `profile_path`, `profile_yaml`, `raw_start`, `route_profile_csv`, `route_profile_path`, `runtime_cfg`, `sim_cfg`, `t`, `tzname`, `value`
- 読み取る主なインスタンス属性: `self._on_speed_cmd`, `self._step`, `self.create_publisher`, `self.create_subscription`, `self.create_timer`, `self.declare_parameter`, `self.df`, `self.forecast_csv`, `self.get_logger`, `self.get_parameter`, `self.limiter`, `self.model`, `self.publish_rate_hz`, `self.set_parameters`, `self.v_exec_kmh`
- 更新する主なインスタンス属性: `self.Tb`, `self.df`, `self.drive_schedule`, `self.exec_time_sec`, `self.forecast_csv`, `self.last_step`, `self.limiter`, `self.model`, `self.pub_alt`, `self.pub_i`, `self.pub_s`, `self.pub_soc`, `self.pub_solar`, `self.pub_speed`, `self.pub_tb`, `self.pub_v`, `self.publish_rate_hz`, `self.route_profile`, `self.s_kmh`, `self.start_time_utc`, `self.timer`, `self.v_cmd_kmh`, `self.v_exec_kmh`, `self.z`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 8、ループ 1、`try` 2

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `super().__init__(...)` を実行する。
2. `self.declare_parameter(...)` を実行する。
3. `self.declare_parameter(...)` を実行する。
4. `self.declare_parameter(...)` を実行する。
5. `self.declare_parameter(...)` を実行する。
6. `self.declare_parameter(...)` を実行する。
7. `self.declare_parameter(...)` を実行する。
8. `self.declare_parameter(...)` を実行する。
9. `self.declare_parameter(...)` を実行する。
10. `self.declare_parameter(...)` を実行する。
11. `self.declare_parameter(...)` を実行する。
12. `self.declare_parameter(...)` を実行する。
13. `self.declare_parameter(...)` を実行する。

14. `self.declare_parameter(...)` を実行する。
15. `self.declare_parameter(...)` を実行する。
16. `self.declare_parameter(...)` を実行する。
17. `self.declare_parameter(...)` を実行する。
18. `self.declare_parameter(...)` を実行する。
19. `self.declare_parameter(...)` を実行する。
20. `self.declare_parameter(...)` を実行する。
21. `self.declare_parameter(...)` を実行する。
22. `self.declare_parameter(...)` を実行する。
23. `self.declare_parameter(...)` を実行する。
24. `self.declare_parameter(...)` を実行する。
25. `self.declare_parameter(...)` を実行する。
26. `self.declare_parameter(...)` を実行する。
27. `self.declare_parameter(...)` を実行する。
28. `profile_yaml` に `str(self.get_parameter('profile_yaml').value or "").strip()` の結果を代入する。
29. 条件 `profile_yaml` を判定し、真なら内部処理を行う。
30. `(profile_path, cfg)` に `load_profile(profile_yaml)` の結果を代入する。
31. `sim_cfg` に `get_section(cfg, 'simulation')` の結果を代入する。
32. `runtime_cfg` に `get_section(cfg, 'runtime')` の結果を代入する。
33. `self.set_parameters(...)` を実行する。
34. `self.publish_rate_hz` に `max(0.2, float(self.get_parameter('publish_rate_hz').value))` の結果を代入する。
35. `self.forecast_csv` に `str(require_csv_data_rows(resolve_path(str(self.get_parameter('forecast_csv').value), 'inputs'), label='forecast', required_columns=('GHI',)))` の結果を代入する。
36. `route_profile_csv` に `str(self.get_parameter('route_profile_csv').value or "").strip()` の結果を代入する。
37. 条件 `route_profile_csv` を判定し、真なら内部処理を行う。
38. `route_profile_path` に `require_csv_data_rows(resolve_path(route_profile_csv, 'inputs'), label='route profile', required_columns=('dist_km',))` の結果を代入する。
39. `self.route_profile` に `pd.read_csv(route_profile_path)` の結果を代入する。
40. 上の条件が偽の場合:
41. `self.route_profile` に `None` の結果を代入する。
42. `drive_schedule_yaml` に `str(self.get_parameter('drive_schedule_yaml').value or "").strip()` の結果を代入する。
43. `self.drive_schedule` に `DriveSchedule.from_yaml(resolve_path(drive_schedule_yaml, 'inputs'))` if `drive_schedule_yaml` else `None` の結果を代入する。
44. `self.df` に `pd.read_csv(self.forecast_csv)` の結果を代入する。
45. 条件 `'time' in self.df.columns` を判定し、真なら内部処理を行う。

46. `t` に `pd.to_datetime(self.df['time'], format='mixed', errors='coerce')` の結果を代入する。
47. `tzname` に `str(self.get_parameter('forecast_time_tz').value or 'UTC')` の結果を代入する。
48. 条件 `t.dt.tz is None` を判定し、真なら内部処理を行う。
49. 条件 `tzname.upper() == 'UTC'` を判定し、真なら内部処理を行う。
50. `t` に `t.dt.tz_localize('UTC')` の結果を代入する。
51. 上の条件が偽の場合:
52. `t` に `t.dt.tz_localize(tzname, ambiguous='NaT', nonexistent='NaT').dt.tz_convert('UTC')` の結果を代入する。
53. 上の条件が偽の場合:
54. `t` に `t.dt.tz_convert('UTC')` の結果を代入する。
55. `self.df['time']` に `t` の結果を代入する。
56. `self.start_time_utc` に `datetime.now(timezone.utc)` の結果を代入する。
57. `raw_start` に `str(self.get_parameter('forecast_start_time_utc').value or '').strip()` の結果を代入する。
58. 条件 `raw_start` を判定し、真なら内部処理を行う。
59. 例外処理を伴う `try` ブロックを実行する。
60. `self.start_time_utc` に `datetime.fromisoformat(raw_start.replace('Z', '+00:00')).astimezone(timezone.utc)` の結果を代入する。
61. `ValueError` を捕捉した場合:
62. `self.get_logger().warning(...)` を実行する。
63. `self.model` に `SolarCarModel(resolve_path(str(self.get_parameter('drive_eff_map').value), 'maps'), resolve_path(str(self.get_parameter('regen_eff_map').value), 'maps'), resolve_path(str(self.get_parameter('rint_map').value), 'maps'), params=Params(dt=1.0/self.publish_rate_hz), panel_eff_map_path=resolve_path(str(self.get_parameter('panel_eff_map').value), 'maps') if str(self.get_parameter('panel_eff_map').value) else None, mppt_eff_map_path=resolve_path(str(self.get_parameter('mppt_eff_map').value), 'maps') if str(self.get_parameter('mppt_eff_map').value) else None, drive_map_eco_path=resolve_path(str(self.get_parameter('drive_map_eco').value), 'maps') if str(self.get_parameter('drive_map_eco').value) else None, drive_map_power_path=resolve_path(str(self.get_parameter('drive_map_power').value), 'maps') if str(self.get_parameter('drive_map_power').value) else None, regen_map_eco_path=resolve_path(str(self.get_parameter('regen_map_eco').value), 'maps') if str(self.get_parameter('regen_map_eco').value) else None, regen_map_power_path=resolve_path(str(self.get_parameter('regen_map_power').value), 'maps') if str(self.get_parameter('regen_map_power').value) else None, ocv_soc_map_path=resolve_path(str(self.get_parameter('ocv_soc_map').value), 'maps') if str(self.get_parameter('ocv_soc_map').value) else None)` の結果を代入する。
64. `params_yaml` に `str(self.get_parameter('params_yaml').value or '').strip()` の結果を代入する。
65. 条件 `params_yaml` を判定し、真なら内部処理を行う。
66. (`_profile_path`, `cfg`) に `load_profile(params_yaml)` の結果を代入する。
67. `model_cfg` に `get_section(cfg, 'model')` の結果を代入する。
68. `model_cfg.items()` を順に走査し、各要素を (`key`, `value`) に入れて処理する。
69. 条件 `hasattr(self.model.p, key)` を判定し、真なら内部処理を行う。
70. 例外処理を伴う `try` ブロックを実行する。
71. `self.v_cmd_kmh` に `float(self.get_parameter('init_speed_kmh').value)` の結果を代入する。
72. `self.v_exec_kmh` に `float(self.get_parameter('init_speed_kmh').value)` の結果を代入する。
73. `self.s_km` に `float(self.get_parameter('s0_km').value)` の結果を代入する。

74. self.z に float(self.get_parameter('soc0').value) の結果を代入する。
75. self.Tb に float(self.get_parameter('Tb0').value) の結果を代入する。
76. self.last_step に None の結果を代入する。
77. self.exec_time_sec に 0.0 の結果を代入する。
78. self.limiter に SmoothRateLimiter(min_value=0.0, max_value=140.0, tau_sec=float(self.get_parameter('filter_tau_sec').value), rise_rate=float(self.get_parameter('accel_limit_kmhps').value), fall_rate=float(self.get_parameter('decel_limit_kmhps').value), initial_value=self.v_exec_kmh) の結果を代入する。
79. self.limiter.reset(...) を実行する。
80. self.pub_speed に self.create_publisher(Float32, '/vehicle/speed_kmh', 10) の結果を代入する。

代表コード断片

```
def __init__(self) -> None:
    super().__init__("solar_state_node")
    self.declare_parameter("profile_yaml", "")
    self.declare_parameter("forecast_csv", "inputs/forecast_10min.csv")
    self.declare_parameter("route_profile_csv", "")
    self.declare_parameter("drive_schedule_yaml", "")
    self.declare_parameter("drive_eff_map", "")
    self.declare_parameter("regen_eff_map", "")
    self.declare_parameter("rint_map", "")
    self.declare_parameter("panel_eff_map", "")
    self.declare_parameter("mppt_eff_map", "")
    self.declare_parameter("drive_map_eco", "")
    self.declare_parameter("drive_map_power", "")
    self.declare_parameter("regen_map_eco", "")
    self.declare_parameter("regen_map_power", "")
    self.declare_parameter("ocv_soc_map", "")
    self.declare_parameter("params_yaml", "")
    self.declare_parameter("forecast_time_mode", "auto")
    self.declare_parameter("forecast_time_tz", "UTC")
    self.declare_parameter("forecast_start_time_utc", "")
    self.declare_parameter("publish_rate_hz", 2.0)
    self.declare_parameter("init_speed_kmh", 45.0)
    self.declare_parameter("soc0", 0.95)
    self.declare_parameter("Tb0", 30.0)
    self.declare_parameter("s0_kmh", 0.0)
    self.declare_parameter("filter_tau_sec", 1.0)
    self.declare_parameter("accel_limit_kmhps", 1.2)
    self.declare_parameter("decel_limit_kmhps", 3.5)

    profile_yaml = str(self.get_parameter("profile_yaml").value or "").strip()
    if profile_yaml:
        profile_path, cfg = load_profile(profile_yaml)
        sim_cfg = get_section(cfg, "simulation")
        runtime_cfg = get_section(cfg, "runtime")
        self.set_parameters(
...

```

7.3 L180 関数 SolarStateNode._on_speed_cmd

- 行範囲: L180–L181
- 所属: SolarStateNode
- 定義: _on_speed_cmd(self, msg: Float32) -> None

- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: self.v_cmd_kmh
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. self.v_cmd_kmh に float(msg.data) の結果を代入する。

代表コード断片

```
def _on_speed_cmd(self, msg: Float32) -> None:
    self.v_cmd_kmh = float(msg.data)
```

7.4 L183 関数 SolarStateNode._forecast_row

- 行範囲: L183–L189
- 所属: SolarStateNode
- 定義: _forecast_row(self, now_utc: datetime)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: any, clip, datetime64, int, len, max, notna, searchsorted
- 戻り値の要点: self.df.iloc[idx] / self.df.iloc[idx]
- 主なローカル代入名: idx
- 読み取る主なインスタンス属性: self.df, self.exec_time_sec, self.publish_rate_hz
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件'time' in self.df.columns and self.df['time'].notna().any() を判定し、真なら内部処理を行う。
2. idx に `int(np.searchsorted(self.df['time'].values, np.datetime64(now_utc)) - 1)` の結果を代入する。
3. idx に `int(np.clip(idx, 0, len(self.df) - 1))` の結果を代入する。
4. self.df.iloc[idx] を返す。
5. idx に `int(np.clip(int(self.exec_time_sec / max(1.0, 3600.0 / self.publish_rate_hz)), 0, len(self.df) - 1))` の結果を代入する。
6. self.df.iloc[idx] を返す。

代表コード断片

```
def _forecast_row(self, now_utc: datetime):
    if "time" in self.df.columns and self.df["time"].notna().any():
        idx = int(np.searchsorted(self.df["time"].values, np.datetime64(now_utc)) - 1)
        idx = int(np.clip(idx, 0, len(self.df) - 1))
        return self.df.iloc[idx]
    idx = int(np.clip(int(self.exec_time_sec / max(1.0, 3600.0 / self.publish_rate_hz)),
0, len(self.df) - 1))
    return self.df.iloc[idx]
```

7.5 L191 関数 SolarStateNode._route_value

- 行範囲: L191–L197
- 所属: SolarStateNode
- 定義: `_route_value(self, field:str, default:float)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float, interpolate_profile`
- 戻り値の要点: `float(default) / float(interpolate_profile(self.route_profile, self.s_km, field, default)) / float(default)`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.route_profile, self.s_km`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `self.route_profile is None` を判定し、真なら内部処理を行う。
2. `float(default)` を返す。
3. 例外処理を伴う `try` ブロックを実行する。
4. `float(interpolate_profile(self.route_profile, self.s_km, field, default))` を返す。
5. `Exception` を捕捉した場合:
6. `float(default)` を返す。

代表コード断片

```
def _route_value(self, field: str, default: float) -> float:
    if self.route_profile is None:
        return float(default)
    try:
        return float(interpolate_profile(self.route_profile, self.s_km, field, default))
    except Exception:
        return float(default)
```

7.6 L199 関数 `SolarStateNode._step`

- 行範囲: L199–L262
- 所属: `SolarStateNode`
- 定義: `_step(self) -> None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Float32`, `_forecast_row`, `_route_value`, `abs`, `clip`, `electrical_balance`, `float`, `get`, `get_clock`, `is_drive_time`, `max`, `now`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `alt_m`, `aux_power_w`, `current_a`, `drive_now`, `dt`, `ghi`, `headwind`, `heat_gain`, `now_sec`, `now_utc`, `out`, `p_pack`, `row`, `slope_pct`, `solar_w`, `tamb`, `tcell`, `voltage_v`
- 読み取る主なインスタンス属性: `self.Tb`, `self._forecast_row`, `self._route_value`, `self.drive_schedule`, `self.exec_time_sec`, `self.get_clock`, `self.last_step`, `self.limiter`, `self.model`, `self.pub_alt`, `self.pub_i`, `self.pub_s`, `self.pub_soc`, `self.pub_solar`, `self.pub_speed`, `self.pub_tb`, `self.pub_v`, `self.publish_rate_hz`, `self.s_km`, `self.start_time_utc`, `self.v_cmd_kmh`, `self.v_exec_kmh`, `self.z`
- 更新する主なインスタンス属性: `self.Tb`, `self.exec_time_sec`, `self.last_step`, `self.s_km`, `self.v_exec_kmh`, `self.z`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. now_sec に self.get_clock().now().nanoseconds / 1000000000.0 の結果を代入する。
2. 条件 self.last_step is None を判定し、真なら内部処理を行う。
3. self.last_step に now_sec の結果を代入する。
4. dt に max(1.0 / self.publish_rate_hz, now_sec - self.last_step) の結果を代入する。
5. self.last_step に now_sec の結果を代入する。
6. self.exec_time_sec を Add で更新する。
7. self.v_exec_kmh に float(self.limiter.update(self.v_cmd_kmh, now=self.exec_time_sec)) の結果を代入する。
8. self.s_kmh を Add で更新する。
9. now_utc に self.start_time_utc + timedelta(seconds=self.exec_time_sec) の結果を代入する。
10. row に self._forecast_row(now_utc) の結果を代入する。
11. ghi に float(row.get('GHI', 0.0)) の結果を代入する。
12. tcell に float(row.get('Tcell_C', 40.0)) の結果を代入する。
13. tamb に float(row.get('Tamb_C', 30.0)) の結果を代入する。
14. headwind に float(row.get('headwind_ms', self._route_value('headwind_ms', 0.0))) の結果を代入する。
15. slope_pct に self._route_value('slope_pct', float(row.get('slope_pct', 0.0))) の結果を代入する。
16. alt_m に self._route_value('alt_m', self._route_value('altitude_m', 0.0)) の結果を代入する。
17. drive_now に self.drive_schedule.is_drive_time(now_utc) if self.drive_schedule is not None else True の結果を代入する。
18. aux_power_w に self.model.scheduled_auxiliary_power(is_driving=drive_now, irradiance_wm2=ghi) の結果を代入する。
19. out に self.model.electrical_balance(self.v_exec_kmh / 3.6, slope_pct, self.z, self.Tb, ghi, tcell, headwind_ms=headwind, aux_power_w=aux_power_w, ambient_temp_c=tamb, elevation_m=alt_m) の結果を代入する。
20. p_pack に float(out['P_pack']) の結果を代入する。
21. current_a に float(out['I']) の結果を代入する。
22. voltage_v に float(out['V']) の結果を代入する。
23. solar_w に float(out['P_pv']) の結果を代入する。
24. self.z に float(np.clip(self.model.soc_step(self.z, p_pack, dt, current_a=current_a, Tbat_C=self.Tb), self.model.p.soc_min, self.model.p.soc_max)) の結果を代入する。
25. heat_gain に 0.015 * abs(current_a) + 0.002 * max(0.0, p_pack / 100.0) の結果を代入する。
26. self.Tb を Add で更新する。
27. self.pub_speed.publish(...) を実行する。
28. self.pub_s.publish(...) を実行する。
29. self.pub_soc.publish(...) を実行する。
30. self.pub_tb.publish(...) を実行する。

31. self.pub_i.publish(...) を実行する。
32. self.pub_v.publish(...) を実行する。
33. self.pub_solar.publish(...) を実行する。
34. self.pub_alt.publish(...) を実行する。

代表コード断片

```
def _step(self) -> None:
    now_sec = self.get_clock().now().nanoseconds / 1e9
    if self.last_step is None:
        self.last_step = now_sec
    dt = max(1.0 / self.publish_rate_hz, now_sec - self.last_step)
    self.last_step = now_sec
    self.exec_time_sec += dt
    self.v_exec_kmh = float(self.limiter.update(self.v_cmd_kmh, now=self.exec_time_sec))
    self.s_km += self.v_exec_kmh * (dt / 3600.0)

    now_utc = self.start_time_utc + timedelta(seconds=self.exec_time_sec)
    row = self._forecast_row(now_utc)
    ghi = float(row.get("GHI", 0.0))
    tcell = float(row.get("Tcell_C", 40.0))
    tamb = float(row.get("Tamb_C", 30.0))
    headwind = float(row.get("headwind_ms", self._route_value("headwind_ms", 0.0)))
    slope_pct = self._route_value("slope_pct", float(row.get("slope_pct", 0.0)))
    alt_m = self._route_value("alt_m", self._route_value("altitude_m", 0.0))

    drive_now = self.drive_schedule.is_drive_time(now_utc) if self.drive_schedule is not
None else True
    aux_power_w = self.model.scheduled_auxiliary_power(
        is_driving=drive_now,
        irradiance_wm2=ghi,
    )
    out = self.model.electrical_balance(
        self.v_exec_kmh / 3.6,
        slope_pct,
        self.z,
        self.Tb,
        ghi,
        tcell,
        headwind_ms=headwind,
        aux_power_w=aux_power_w,
        ambient_temp_c=tamb,
        elevation_m=alt_m,
    )
    ...
```

7.7 L265 関数 main

- 行範囲: L265–L270
- 所属: module 直下
- 定義: main()->None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: SolarStateNode, destroy_node, init, shutdown, spin
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: node

- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `rclpy.init(...)` を実行する。
2. `node` に `SolarStateNode()` の結果を代入する。
3. `rclpy.spin(...)` を実行する。
4. `node.destroy_node(...)` を実行する。
5. `rclpy.shutdown(...)` を実行する。

代表コード断片

```
def main() -> None:
    rclpy.init()
    node = SolarStateNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

7.8 設定値と入力パラメータ

行	名前	既定値・補足
25	<code>profile_yaml</code>	
26	<code>forecast_csv</code>	<code>inputs/forecast_10min.csv</code>
27	<code>route_profile_csv</code>	
28	<code>drive_schedule_yaml</code>	
29	<code>drive_eff_map</code>	
30	<code>regen_eff_map</code>	
31	<code>rint_map</code>	
32	<code>panel_eff_map</code>	
33	<code>mppt_eff_map</code>	
34	<code>drive_map_eco</code>	
35	<code>drive_map_power</code>	
36	<code>regen_map_eco</code>	
37	<code>regen_map_power</code>	
38	<code>ocv_soc_map</code>	
39	<code>params_yaml</code>	
40	<code>forecast_time_mode</code>	<code>auto</code>
41	<code>forecast_time_tz</code>	<code>UTC</code>

42	forecast_start_time_utc	
43	publish_rate_hz	2.0
44	init_speed_kmh	45.0
45	soc0	0.95
46	Tb0	30.0
47	s0_km	0.0
48	filter_tau_sec	1.0
49	accel_limit_kmhps	1.2
50	decel_limit_kmhps	3.5

7.9 ROS topic I/O

行	種別	topic	callback / 補足
168	publisher	/vehicle/speed_kmh	publisher
169	publisher	/vehicle/s_km	publisher
170	publisher	/vehicle/batt_soc	publisher
171	publisher	/vehicle/batt_temp_c	publisher
172	publisher	/vehicle/batt_current_a	publisher
173	publisher	/vehicle/batt_voltage_v	publisher
174	publisher	/vehicle/solar_power_w	publisher
175	publisher	/vehicle/altitude_m	publisher
176	subscription	/planner/speed_cmd	self._on_speed_cmd

8 処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/solar_state_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。